

INSTITUTO POLITÉCNICO DE LEIRIA

Fundamentos de Programação

TeSP PSI | 1º ano

Relatório de protótipo de aplicação

Diogo Henriques Faria Nº 2241583

José Miguel Coelho Dias Nº 2240099

Trabalho de Projeto da unidade curricular de Fundamentos de Programação

Leiria, 01 de 2025

1. Resumo

Neste projeto implementamos um projeto onde os alunos possam submeter os exercícios realizados durante as aulas práticas.

Para a realização deste projeto utilizamos a linguagem de programação *C* para fazer todo o controlo das funções e todas as funcionalidades to projeto.

Na realização deste projeto utilizamos um software chamado *Visual Studio Code* para a programação toda do projeto

Índice

1. Resumo *2*

2. Introdução *5*

3. Desenvolvimento *6*

3.1. Estruturas de dados utilizadas (vetores, *structs*, ficheiros) 6

3.1.1. Vetores: 6

3.1.2. Structs: 7

3.1.3. Ficheiros: 7

3.2. Constantes e variáveis principais 8

3.3. Funcionalidades implementadas e respetivas funções 9

3.3.1. Funções de menus 9

3.3.2. Funções de estudantes 10

3.3.3. Funções de fichas 10

3.3.4. Funções de exercícios 11

3.3.5. Funções de submissões 12

3.3.6. Funções de estatísticas 12

3.4. Funcionalidades por implementar 13

3.5. Validação de dados de entrada 14

4. Cenário de Teste *15*

4.1. Objetivo 15

4.2. Teste registar estudantes 15

4.3. Teste registar fichas 16

4.4. Teste registar exercícios 17

4.5. Teste registar submissões 18

4.6. Teste Guardar e ler ficheiro 19

5. Conclusão *20*

Índice de Figuras

Figura 1- Vetores 6

Figura 2-Algumas structs no projeto 7

Figura 3-Constantes do projeto 8

Figura 4-Funções de menus 9

Figura 5- Funções das fichas 10

Figura 6- Funções de exercícios 11

Figura 7- Funções das estatísticas 12

Figura 8- Funções para validações e funcionalidades 14

2. Introdução

Neste seguinte projeto, desenvolvemos um protótipo de aplicação em linguagem C que permite-nos gerir a informação relacionada com as submissões de exercícios feitos nas aulas práticas. Este protótipo é feito para uma avaliação curricular de Fundamentos de Programação e está alinhada com os requisitos do projeto OPSA 2.0 (Observatório para o Sucesso Académico) onde tem como fim, promover o acompanhamento dos estudantes e prevenir o abandono escolar.

O protótipo foi pensado de modo a o utilizador poder registar e consultar vários tipos de dados, como os estudantes, as fichas, os exercícios de cada ficha e as submissões feitas. Damos como exemplo, que o sistema deve requerer ao utilizador no momento de registar um aluno o seu número de estudante verificar se já existe e de seguida caso não exista prosseguir o registo do seu nome e *email*.

Este trabalho foi uma oportunidade para praticar conceitos importantes de programação, como o uso de estruturas de repetição no acesso a *arrays*, estruturas de dados, manipulação de *strings* e como guardar e ler ficheiros.

Além disso, foi pensado para ser fácil, interativo e simples de usar com menus simples e mensagens claras. No futuro, este protótipo pode servir como base para soluções mais completas e automatizadas no âmbito do sucesso académico.

3. Desenvolvimento

3.1. Estruturas de dados utilizadas (vetores, *structs*, ficheiros)

3.1.1. Vetores:

Foram usados para conseguir armazenar informações sobre estudantes, fichas, exercícios e submissões.

Exemplos incluem:

- t_estudante estudantes [MAX_ESTUDANTES]: Um vetor que armazena até 100 estudantes.
- t_fichas ficha [MAX_FICHAS]: Um vetor que armazena até 10 fichas de exercícios.
- t_submissoes submissao [MAX_SUBMISSOES]: Um vetor que armazena até 10.000 submissões.

```
t_estudante estudantes[MAX_ESTUDANTES];
t_submissoes submissao[MAX_SUBMISSOES];
t_fichas ficha[MAX_FICHAS];
```

Figura 1- Vetores

3.1.2. Structs:

Utilizamos estruturas que contêm campos para guardar informações específicas, como IDs únicos, nomes e detalhes específicos escolares e datas.

- t_data: Usada para guardar data com dia, mês e ano.
- t_fichas: Representa uma ficha de exercícios, onde temos um identificador único, nome, número de exercícios, data de publicação e um vetor de t_exercícios.
- t_exercicios: Representa um dos exercícios que pertencem a t_fichas, incluído com um identificador único, nome, identificador da ficha, grau de dificuldade e tipo de solução.
- t_estudante: Trata-se de um estudante, que nele tem um identificador único, número de estudante, nome e *email*.
- t_submissoes: Representa uma submissão de exercício, onde inclui um identificador único, identificadores de estudante, ficha e exercício, data de submissão, número de linhas e classificação obtida.

```
14  typedef struct
15  {
16      int dia, mes, ano;
17  } t_data;
18
19  typedef struct
20  {
21      int identificador_unico_exercicios;
22      char nome_exercicio[MAX_CARACTERES];
23      int identificador_unico_ficha;
24      char grau_de_dificuldade[MAX_CARACTERES];
25      char tipo_de_solução[MAX_CARACTERES];
26
27  } t_exercicios;
28
29  typedef struct
30  {
31      int identificador_unico_ficha;
32      char nome_da_ficha[MAX_CARACTERES];
33      int numero_de_exercicios;
34      t_data data_de_publicacao;
35      t_exercicios exercicio[MAX_EXERCICIOS];
36
37  } t_fichas;
```

Figura 2-Algumas structs no projeto

3.1.3. Ficheiros:

Os dados do protótipo são gravados em modo de acesso “wb” e lidos em “rb” num arquivo binário denominado de (dados.bin).

3.2. Constantes e variáveis principais

Foram usadas constantes para definir os limites seja para número de estudantes, fichas, exercícios, submissões e o tamanho máximo das *strings* tais como:

- MAX_ESTUDANTES,
- MAX_FICHAS,
- MAX_EXERCICIOS,
- MAX_SUBMISSOES
- MAX_CARACTERES

Nas variáveis usamos contadores que atualizam o número atual de estudantes, fichas e submissões como: “num_estudantes”, “num_fichas” e “num_submissoes”. As variáveis “estudantes”, “ficha”, “submissão” armazenam os dados em memória e por fim “menu” e “opcao” foram usadas para orientar o sistema da direção do utilizador durante a interação com os menus.

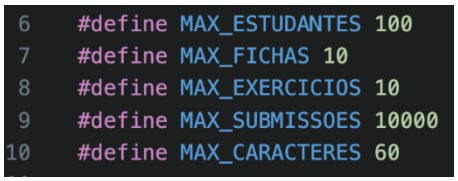


Figura 3-Constantes do projeto

3.3. Funcionalidades implementadas e respetivas funções

3.3.1. Funções de menus

As funções do menu do protótipo são responsáveis por apresentar opções ao usuário e permitir que ele lide com o sistema de maneira fluida e facilitada.

- Menu_opcoes: É o menu principal onde podemos aceder a submenus de estudantes, exercícios, fichas submissões ainda nos permite gravar e ler ficheiro de dados.
- Display_menus: Esta função e um submenu geral onde usamos para diferentes propósitos seja no menu de estudantes ou até de submissões.
- Menu_estudantes: É o menu que nos permite registar e consultar dados do estudante.
- Menu_fichas: É o menu que nos permite registar e consultar dados das fichas.
- Menu_exercicios: É o menu que nos permite registar e consultar dados do exercício.
- Menu_Submissoes: É o menu que nos permite registar e consultar dados das submissões.
- Menu_estatisticas: Usado para obtermos total de submissões por estudante, media de classificações e percentagem de exercício.

```
59  // MENUS
60  int menu_opcoes();
61  int display_menus(char tipo_menu[]);
62  int menu_estudantes(t_estudante estudantes[], int numero_estudantes);
63  int menu_fichas(int num_fichas, t_fichas ficha[]);
64  int menu_exercicios(t_fichas ficha[], int num_fichas);
65  int display_estastisticas();
66  void menu_estatisticas(t_estudante estudantes[], t_fichas ficha[], t_submissoes submissao[], int num_submissoes, int numero_estudantes);
67  int menu_submissoes(t_estudante estudantes[], t_submissoes submissao[], t_fichas ficha[], int numero_submissoes, int numero_estudantes, int
68  char confirmar_saida();
```

Figura 4-Funções de menus

3.3.2. Funções de estudantes

As funções de estudantes são responsáveis por registar e consultar os dados dos estudantes.

- Registar_estudante: Função responsável por registar um novo estudante no sistema.
- Procurar_estudante: Função responsável por procurar um estudante no sistema.
- Mostrar_dados_estudante: Função responsável por mostrar os dados dos estudantes registados no sistema.

3.3.3. Funções de fichas

As funções de estudantes são responsáveis por registar e consultar os dados das fichas.

- Registar_ficha: Função responsável por registar uma nova ficha no sistema.
- Reunir_Dados_fichas: Função responsável por reunir os dados da nova ficha a ser registada.
- Mostrar_dados_fichas: Função responsável por exibir todos os dados das fichas do sistema.
- Dados_fichas: Funcao que percorre o vetor fichas e mostra os dados de cada ficha.
- Procurar_ficha: Função responsável por procurar uma ficha no sistema.

```
74  // FICHAS
75  int registrar_ficha(int num_fichas, t_fichas ficha[]);
76  void reunir_dados_fichas(int num_fichas, t_fichas ficha[], int ficha_requerida);
77  void mostrar_dados_fichas(int num_fichas, t_fichas ficha[]);
78  void dados_fichas(int num_fichas, t_fichas ficha[]);
79  int procurar_ficha(int num_fichas, t_fichas ficha[], int ficha_requerida);
```

Figura 5- Funções das fichas

3.3.4. Funções de exercícios

As funções de estudantes são responsáveis por registar e consultar os dados de exercícios.

- Registrar_exercicios: Função responsável pelo registo de novos exercícios em uma ficha.
- Reunir_dados_exercicios: Função que reúne os dados necessários para o registo.
- Grau_dificuldade: Função com menu que solicita o grau de dificuldade.
- Tipo_de_solucao: Função com menu que solicita o tipo de solução.
- Mostrar_dados_exercicios: Função que mostra os dados de uma ficha.
- Dados_exercicios: Função responsável por percorrer o vetor de exercícios de uma ficha.
- Procurar_exercicios: Função responsável por procurar um exercício específico.



```
80  // EXERCICIOS
81
82  void registrar_exercicios(int num_fichas, t_fichas ficha[]);
83  void reunir_dados_exercicios(int num_fichas, t_fichas ficha[], int id_ficha);
84  void grau_dificuldade(char grau[]);
85  void tipo_de_solucao(char solucao[]);
86  void mostrar_dados_exercicios(int num_fichas, t_fichas ficha[]);
87  void dados_exercicios(int num_fichas, t_fichas ficha[], int id_ficha);
88  int procurar_exercicio(int num_fichas, t_fichas ficha[], int indice_ficha, int exercicio_requerido);
89
```

Figura 6- Funções de exercícios

3.3.5. **Funções de submissões**

As funções de estudantes são responsáveis por registar e consultar os dados das submissões.

- Registar_submissao: Função responsável por registar uma nova submissão de exercício feita por um estudante.
- Reunir_dados_submissao: Função que reúne dados para registar uma nova submissão.
- Mostrar_dados_submissao: Função que exibe todas as submissões registadas no sistema.

3.3.6. Funções de estatísticas

As funções de estatísticas são responsáveis por calcular alguns dados e retornar estatísticas ao utilizador sobre os dados inseridos por ele.

- Total_submissoes_estudante: Função responsável por calcular o total de submissões feitas por um estudante específico.
- Media_classificacao: Função que calcula a media das classificações obtidas por um estudante.
- Percentagem_exercicios: Função que calcula a percentagem de exercícios submetidos por um estudante em relação ao total de exercícios das fichas.

```
95  // ESTATISTICAS
96  int total_submissoes_estudante(t_submissoes submissao[], t_estudante estudantes[], int num_submissoes, int estudante_id);
97  float media_classificacao(t_submissoes submissao[], t_estudante estudantes[], int num_submissoes, int estudante_id);
98  float percentagem_exercicios(t_submissoes submissao[], t_estudante estudantes[], t_fichas ficha[], int num_submissoes, int estudante_id);
```

Figura 7- Funções das estatísticas

3.4. Funcionalidades por implementar

O código que foi criado ainda é só um protótipo no qual ainda poderiam ser adicionadas mais funcionalidades e aprimoradas as presentes:

Melhorar o *design* gráfico: No presente, o sistema corre em uma interface de linha de comando, assim ainda seria possível melhorar o desenho, o que ajudaria na fluidez e na rápida aprendizagem de uso por parte do utilizador.

Sistema de *Login*: Poderíamos implementar um sistema hierárquico com diferentes níveis de acesso desde o administrador até ao aluno com algumas permissões em comum e outras restritas para aceder os utilizadores precisavam ser autorizados.

Feedback: Depois de criado o sistema de *login* tanto o aluno poderia deixar a sua opinião sobre as fichas, no que teve dificuldade como o professor poderia deixar um *feedback* dos seus erros e como melhorar de modo a prevenir o sentimento de abandono escolar.

3.5. Validação de dados de entrada

De modo a prevenir que os dados inseridos pelo utilizador no sistema fossem os mais restritos, próximos do correto e do que é pedido tomamos em conta assim criar algumas funções que fizessem a validação desses dados.

Começamos pelos de entrada como ler um número tanto inteiro como decimal, onde incluímos uma mensagem, um mínimo e um máximo para podemos reutilizar esta função as vezes que fossem precisas. De seguida avançámos para 3 funções que iriam ser cruciais no desenvolvimento do protótipo são elas "id_estudante", "id_ficha" e "Id_exercicio", tem como objetivo receber o número procurado seja de estudante, ficha ou exercício e devolver se existe, caso exista devolve a posição do *array* para pudermos mostrar ou registar dados.

No registo de dados em duas partes do projeto tanto na ficha como na submissão é pedido uma data, tivemos o cuidado de com 3 funções que funcionam em conjunto validar se o ano é bissexto tendo em conta os dias de fevereiro, os meses com 30 e 31 dias.

Por fim não com funções, mas dentro das funções gravar e ler ficheiro incluímos um *if* em cada, caso o ficheiro seja diferente de "*NULL*" emitir uma mensagem de erro.

```
100  // VALIDACOES E FUNCIONALIDADES
101  int ler_numero_inteiro(char mensagem[], int minimo, int maximo);
102  float ler_numero_decimal(char mensagem[], int minimo, int maximo);
103  int id_estudante(t_estudante estudantes[], int numero_estudantes);
104  int id_ficha(t_fichas ficha[], int num_fichas);
105  int id_exercicio(t_fichas ficha[], int num_fichas, int id_ficha);
106  int validador_ano();
107  int validador_mes();
108  int validador_dia(int mes, int ano);
109  void gravar_ficheiro_binario(t_estudante estudantes[], int numero_estudantes);
110  int ler_ficheiro(t_estudante estudantes[]);
```

Figura 8- Funções para validações e funcionalidades

4. Cenário de Teste

4.1. Objetivo

No tópico em que nos encontramos temos como objetivo encenar uma possível utilização do protótipo criado na Unidade Curricular Fundamentos Da Programação por parte do utilizador. Numa primeira utilização o utilizador deve ter o seu protótipo inicializado e o sistema encontrar-se sem dados e com as suas estruturas de dados limpas e variáveis bem inicializadas, o que é importante para um funcionamento correto do código.

4.2. Teste registar estudantes

1. Executar o protótipo;
2. Selecionar a opção em menu “Registar e consultar dados de um estudante”;
3. Selecionar a opção em menu “Registar estudantes”;
4. Inserir o número de estudante;
5. A função “id_estudante” valida se o número de estudante existe;
6. Se existir, mostra uma mensagem de aviso “O estudante que tentou registar já existe”;
7. Caso contrário prossegue e requisita o nome do estudante e atribui um *email*;
8. Guardados os seus dados incrementa mais um no número de estudantes;
9. Selecionar a opção em submenu “estudantes" consultar dados de estudantes;
10. Verificar se os dados do estudante registado são exibidos corretamente.

4.3. Teste registar fichas

1. Executar o protótipo;
2. Selecionar a opção em menu “Registar e consultar dados de ficha”;
3. Selecionar a opção em menu “Registar ficha”;
4. Inserir o número de ficha;
5. A função “registar_ficha” valida se o número de fichas registadas é menor que o máximo de fichas, caso não imprime mensagem de erro de contrário prossegue;
6. Chama a função “reunir_dados_fichas” onde requisita os dados da ficha:
    a. Nome;
    b. Número de exercícios;
    c. Data de publicação.
7. Guardados os seus dados incrementa mais um no número de fichas;
8. Selecionar a opção em submenu “fichas” consultar dados de fichas;
9. Verificar se os dados de fichas registadas são exibidos corretamente.

4.4. Teste registar exercícios

1. Executar o protótipo;
2. Certificar que já existe uma ficha registada;
3. Selecionar a opção em menu “Registar exercício”;
4. Inserir o número da ficha existente caso não exista imprime uma mensagem de erro de contrário prossegue o registo;
5. A função “id_ficha” valida se o número de ficha existe e armazenamos o primeiro id dessa ficha numa variável se for diferente de 0 sinónimo que já existe esse exercício e imprime mensagem de erro caso contrário prossegue;
6. Chama “reunir_dados_exercicios” onde registamos os dados em modo menu:
    a. Tipo de solução;
    b. Grau dificuldade.
7. Guardados os seus dados;
8. Selecionar a opção em submenu “exercícios” consultar dados de exercícios;
9. Verificar se os dados do exercício registado são exibidos corretamente.

4.5. Teste registar submissões

1. Executar o protótipo;
2. Certificar que já existe um estudante, ficha e exercício registados;
3. Selecionar a opção em menu “Registar submissão”;
4. Inserir o número de estudante, função “id_estudante” valida se o estudante existe;
5. Inserir o número de ficha, função “id_ficha” valida se a ficha existe;
6. Inserir o número do exercício, função “id_exercicio” valida se o exercício existe;
7. Caso alguma das informações não tenha sido registada o sistema emite uma mensagem de erro caso contrário prossegue;
8. Chama a função “reunir_dados_submissao” e onde inserimos os dados:
    a. Data de submissão;
    b. Número de linhas;
    c. Classificação do exercício.
9. Guardados os seus dados incrementa mais um no número de submissões;
10. Selecionar a opção no submenu submissões para consultar dados de submissões;
11. Verificar se os dados das submissões registadas são exibidos corretamente.

4.6. Teste Guardar e ler ficheiro

1. Executar o protótipo;
2. Certificar que já existe um estudante, ficha, exercício ou submissão registados;
3. Selecionar a opção em menu “Gravar dados em ficheiro”;
4. Abre o ficheiro “dados.bin” em modo *wb* onde cria um ficheiro binário para operações de escrita;
5. Se o ficheiro for diferente de “NULL” escreve os dados e mostra mensagem de sucesso caso contrário mostra mensagem de erro;
6. Fecha o ficheiro;
7. Sair do protótipo;
8. Executar o protótipo;
9. Selecionar a opção em menu “Ler dados de ficheiro”;
10. Abre o ficheiro “dados.bin” em modo *rb* onde abre um ficheiro binário para operações de leitura;
11. Se o ficheiro for diferente de “*NULL*” lê os dados e mostra mensagem de sucesso caso contrário mostra mensagem de erro;
12. Selecionar uma opção no menu e consultar se dados anteriormente foram gravados;
13. Verificar se os dados registados são exibidos corretamente.

5. Conclusão

Ao finalizar este projeto, é gratificante constatar que conseguimos alcançar com sucesso os objetivos estabelecidos no início do projeto. Tínhamos como missão conseguir melhorar as nossas capacidades na prática da programação e a nossa capacidade lógica. Durante a realização deste projeto houve dificuldades em ultrapassar algumas lacunas que tínhamos mais dificuldade na programação, no entanto com esforço e pesquisa conseguimos ultrapassá-las com sucesso.

Em conclusão ficámos extremamente gratificantes por conseguirmos concluir este projeto com sucesso e mais aprendizagens na prática da programação.